

Computer Programming Summer Interest Group

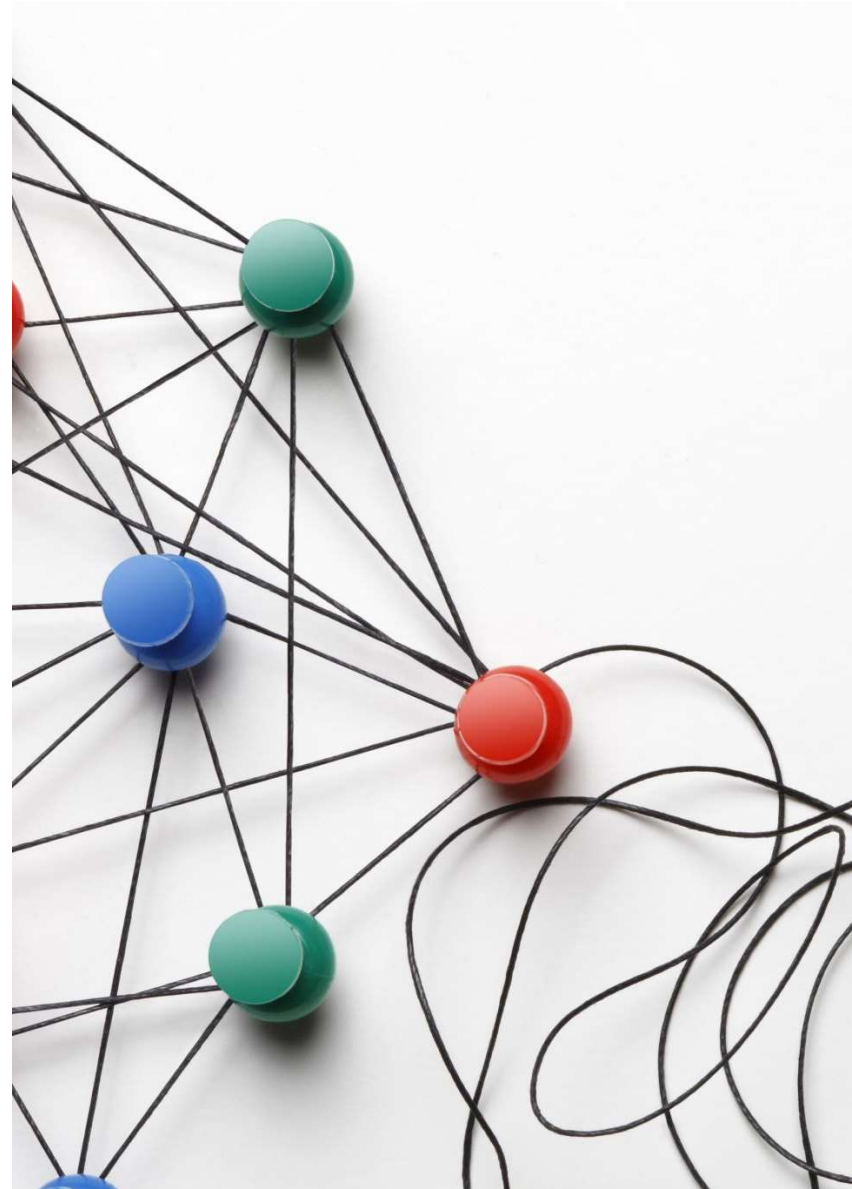
Session 13 / Lambdas

Ephesians 2:4-10

But God, being rich in mercy, because of the great love with which he loved us, even when we were dead in our trespasses, made us alive together with Christ—by grace you have been saved—and raised us up with him and seated us with him in the heavenly places in Christ Jesus, so that in the coming ages he might show the immeasurable riches of his grace in kindness toward us in Christ Jesus. For by grace you have been saved through faith. And this is not your own doing; it is the gift of God, not a result of works, so that no one may boast. For we are his workmanship, created in Christ Jesus for good works, which God prepared beforehand, that we should walk in them.

Objectives

- By the end of this session, you will be able to:
 - Understand the purpose of lambda's
 - Understand the relationship between lambdas and functions
 - Understand the syntax and structure of lambdas
 - Learn how to write and use lambdas



Lambda functions

- Lambda functions are anonymous (they do not require a name).
- Defined using the syntax:
`lambda <arguments>: <expression>`
- They can take any number of arguments, but only one expression.
- Often used with functions like `map()`, `filter()` and `sorted()`.
- Useful for short, concise operations where defining a full function is unnecessary.

Passing a function into a function

- We can define a function that takes a function as an argument
 - Use the type hint “Callable”
 - Specify a list of arguments and return type
- In this case, we pass the “add_ten” function as an argument into “gen_string”
- We use this to generate a list of values derived from a source list

```
from typing import Callable
```

```
def add_ten(x: float) -> float:  
    return x + 10.0
```

```
def gen_string(value: float,  
               adder: Callable[[float], float]) -> str:  
    return f"{value} + {adder(0.0)} = {adder(value)}"
```

```
values = [5.0, 16.0, 27.0, 38.0, 49.0]  
strings = [  
    gen_string(value=x, adder=add_ten)  
    for x in values]
```

```
for string in strings:  
    print(string)
```

Using a lambda instead of a function

- It can be inconvenient to define a function which is going to be used only locally
- Here we use a “lambda” which defines the function locally
- We can pass it as an argument just as we could in the previous example

```
from typing import Callable
```

```
def gen_string(value: float,  
              adder: Callable[[float], float]) -> str:  
    return f"{value} + {adder(0.0)} = {adder(value)}"
```

```
values = [5.0, 16.0, 27.0, 38.0, 49.0]  
strings = [  
    gen_string(value=x, adder=lambda x: x + 10.0)  
    for x in values]
```

```
for string in strings:  
    print(string)
```


Use of 'filter' function

- The 'filter' function can be used to filter elements of a collection
- Pass a function as first argument
- The function takes a single argument from the collection
- Values resulting in a True are allowed into the resulting collection

```
def is_odd(x: int) -> bool:  
    return x % 2 == 1
```

```
def odd_values(max_val: int) -> list[int]:  
    return list(filter(is_odd, range(max_val + 1)))
```

```
print(f"Odd values up to 10: {odd_values(10)}")
```

Use of 'filter' function with a lambda

- We can substitute the function with a lambda instead
- The lambda takes a single argument and returns a Boolean value

```
def odd_values(max_val: int) -> list[int]:  
    return list(  
        filter(lambda x: x % 2 == 1,  
              range(max_val + 1)))  
  
print(f"Odd values up to 10: {odd_values(10)}")
```


Using 'map' to change values of a collection

- Similar to filter, we can use map to transform elements of a collection
- Provide a function to transform / map each element in the collection to some other value

```
def square(x: int) -> int:  
    return x**2
```

```
def square_values(max_val: int) -> list[int]:  
    return list(map(square, range(max_val)))
```

```
print(f"Squares in range(10): {square_values(10)}")
```

Using 'map' to change values of a collection with a lambda

- Similar to filter, we can use map to transform elements of a collection
- Provide a function to transform / map each element in the collection to some other value

```
def square_values(max_val: int) -> list[int]:  
    return list(  
        map(lambda x: x**2,  
            range(max_val)))  
  
print(f"Squares in range(10): {square_values(10)}")
```

The “reduce” function to reduce a collection to a single value

- Consider summing the elements of a list
 - We can use a for loop and accumulate the values
- Python provides the “reduce” function accumulate a single value from a collection.
- Need to import “reduce” from the functools package

```
from functools import reduce
```

```
def add(x: int, y: int) -> int:  
    return x + y
```

```
def sum_vals(max_val: int) -> int:  
    return reduce(add,  
                  range(max_val))
```

```
print(f"sum of elements in range(10): {sum_vals(10)}")
```

We can use a lambda in the “reduce” function

- Here again, we can use the lambda in place of a function defined elsewhere in our program

```
from functools import reduce

def sum_vals(max_val: int) -> int:
    return reduce(lambda x, y: x + y, range(max_val))

print(f"sum of elements in range(10): {sum_vals(10)}")
```



Activities

- Use 'map' and 'reduce' to approximate the value of $\frac{\pi^2}{6}$ by summing the values of $\frac{1}{n^2}$ for n in some range from 1 to max_n.
 - Use different values of max_n to see how the values converge
- Use 'map' and 'reduce' to add the sum the values of $\frac{1}{2^n}$ for n in some range from 1 to max_n.
 - What value does the sum converge to as max_n increases?